
PORTFOLIO - GRADUAAT PROGRAMMEREN

Persoonlijk Dashboard

Een webapplicatie met vier API-gedreven widgets

De ontwerper · Doel 4: Je stemt de methodiek en programmeertaal af op de toepassing

Auteur Jonas Trap

Opleiding Graduaat Programmeren, Thomas More Geel

Academiejaar 2025-2026

Type bewijs eigen project

Het project

Dit dashboard is een persoonlijk project: een single-page webapplicatie die vier dagelijkse noden samenbrengt op één centrale pagina. De vier widgets tonen het actuele weerbericht voor opgeslagen locaties, een dagelijks avondmaalrecept met filters voor dieet en bereidingstijd, de NASA-foto van de dag met uitleg, en een eenvoudig takenbeheer.

Het project dient als demonstratiestuk van mijn vaardigheden in webontwikkeling en API-integratie. De nadruk lag bewust op een nette, professionele uitwerking die de geleerde concepten concreet toepast: een duidelijke architectuur, type-veiligheid, integratie met externe API's en degelijke foutafhandeling.

Technische keuzes

- **Build tool:** Vite, voor een snelle ontwikkelomgeving met native TypeScript-ondersteuning.
- **Taal:** TypeScript, voor type-veiligheid en betere onderhoudbaarheid.
- **CSS-framework:** Bootstrap 5, voor een degelijke grid en ingebouwde dark mode.
- **API's:** OpenWeatherMap (weer), Spoonacular (recepten), NASA APOD (foto van de dag), en een vertaal-API voor de Nederlandse inhoud.

Broncode

De volledige broncode van dit project staat publiek op GitHub, zodat ze altijd terug te vinden is: github.com/jonastapmore/jonastap-projecten/tree/main/dashboard

Methodiek en programmeertaal afstemmen

Ik koppel dit project aan dit doel omdat ik de technologie en de werkwijze doelbewust heb afgestemd op wat de toepassing nodig had, en de oplossing onderhoudbaar heb gehouden. Hieronder de twee kernpunten van het doel.

1. Een programmeeromgeving die past bij de oplossing

Het dashboard is een frontend-webapplicatie die meerdere API's combineert. Ik koos een stack en omgeving die daar precies bij passen:

Onderdeel	Keuze	Waarom past dit bij de toepassing
Build tool	Vite	Snelle ontwikkelserver, native TypeScript en een eenvoudige statische build voor de hosting.
Taal	TypeScript	Type-veiligheid en zelfdocumenterende code voor een project dat met meerdere API's groeit.
UI-framework	Bootstrap 5	Kant-en-klare grid en dark mode, zodat een verzorgde, responsieve UI snel klaar is.
Opslag	localStorage	Lichte persistentie die past bij een frontend-only app zonder backend.
Recept-API	Spoonacular	Bewust gekozen na TheMealDB, omdat enkel Spoonacular voedingswaarde en prijs levert voor de filters.

- De architectuur volgt hetzelfde, herkenbare patroon als een eerder portfolio-project, zodat ze aansluit bij een **bestaande oplossing** en werkwijze.
- **Afstemmen op de toepassing** was een doorlopend proces: toen TheMealDB geen voedingswaarde of prijs bleek te hebben voor de filters, stapte ik over naar Spoonacular.
- De **omgeving**: VS Code als editor, git voor versiebeheer, en een statische build die ik met een aangepast basis-pad op de hosting (Combell, submap /dashboard/) heb geplaatst.

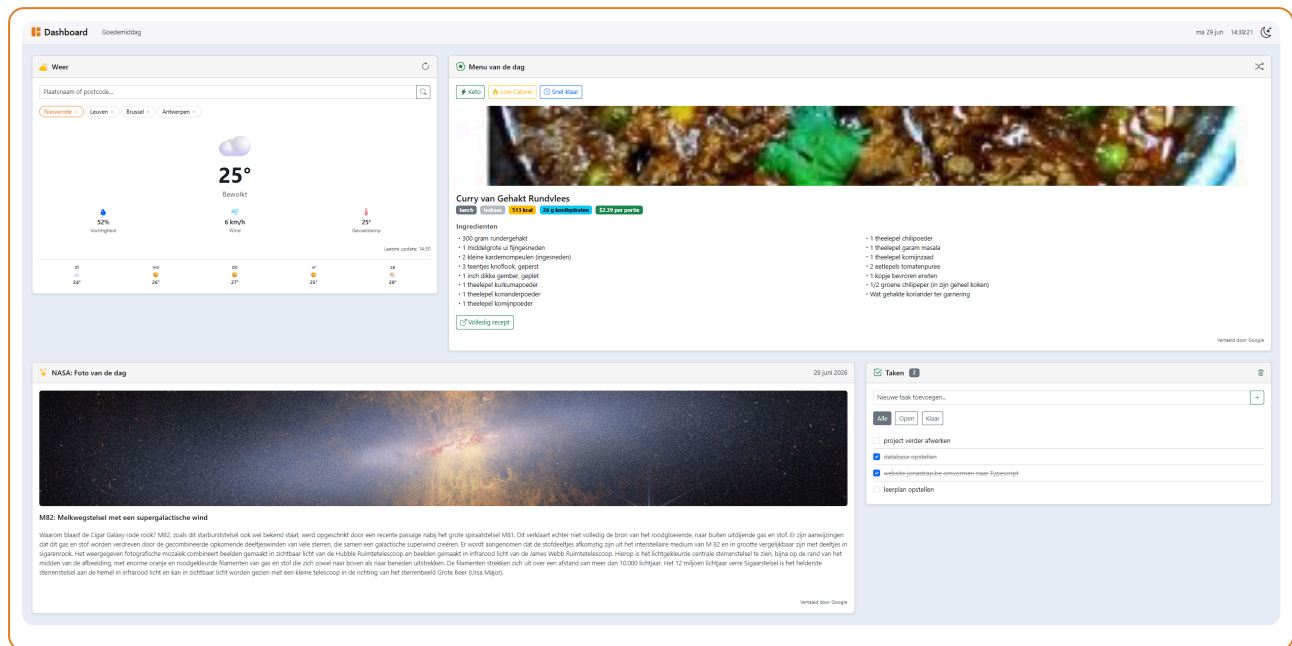
2. Onderhoudbaar voor collega's

Ik bouwde de oplossing zo op dat iemand anders ze vlot kan begrijpen, aanpassen en uitbreiden:

- **Eén consistent patroon** over alle widgets (elk een custom element met dezelfde opbouw), zodat een collega na één widget de rest meteen begrijpt.
- **Getypeerde code**: de interfaces en types maken duidelijk welke data verwacht wordt, wat aanpassen veiliger maakt.

- **Documentatie:** Nederlandse comments doorheen de code en een uitgebreide README (architectuur, technische keuzes, planning).
- **Gescheiden lagen en hergebruik:** aparte mappen per verantwoordelijkheid, een abstracte provider en een gedeelde vertaalhulp, zodat er weinig dubbele code is.
- **Versiebeheer:** git met betekenisvolle commits en een `.gitignore`, zodat het project net en deelbaar blijft.

Het resultaat



Afbeelding 1: het dashboard, gebouwd met de gekozen stack (Vite, TypeScript, Bootstrap).

Een vaste, onderhoudbare structuur per widget

Elke widget volgt dezelfde opbouw met sprekende methodenamen. Een collega die één widget leest, begrijpt meteen de andere:

```
export class RecipeWidget extends CustomElement {
  constructor() {
    super(HTML);
    // luisteraars koppelen en meteen een recept laden
  }

  async #getRandomRecipe() { /* haalt een recept op, rekening houdend met de filters */ }
  async #showMeal(recipe: Recipe) { /* vult de kaart met het recept */ }
  async #renderIngredients(recipe: Recipe) { /* bouwt de ingredientenlijst op */ }
  #showError(bericht: string) { /* toont een nette foutmelding */ }
}
```

Codefragment uit `src/components/recipeWidget/recipeWidget.ts`.

Mapstructuur

De duidelijke laagopbouw maakt het project overzichtelijk en onderhoudbaar:

```
dashboard/
src/
  components/  widgets en navbar (custom elements)
  data/        persistence providers
```



effects/	thema-beheer en de gedeelde vertaalhulp
models/	datamodellen (Task, SavedLocation)
pages/	de dashboardpagina
router/	Router, Page en CustomElement basisklassen
main.ts	registratie van de widgets en de routes
style.css	
index.html	
tsconfig.json	
vite.config.ts	

Zelfreflectie

Wat ging goed

De gekozen stack paste goed bij de toepassing: Vite gaf me een snelle ontwikkelomgeving, TypeScript hield de groeiende code beheersbaar, en Bootstrap leverde snel een verzorgde, responsieve interface. Door overal hetzelfde patroon te gebruiken, bleef de oplossing overzichtelijk en uitbreidbaar.

Wat ik geleerd heb

Ik leerde dat afstemmen op de toepassing een doorlopende keuze is, geen eenmalige beslissing. Toen TheMealDB niet voldeed (geen voedingswaarde of prijs voor de filters), heb ik bewust een andere API gekozen die wél bij de eisen paste. Ook besepte ik hoe sterk documentatie en een vaste structuur de onderhoudbaarheid bepalen: comments, types en een README maken het verschil voor wie later met de code werkt.

Uitdagingen

Een afweging was frontend-only werken zonder backend. Dat past bij de schaal van het project, maar betekent ook dat API-sleutels in de browser belanden. Voor de hosting moest ik de omgeving afstemmen op de submap (/dashboard/) door het basis-pad in de build aan te passen, anders vond de browser de bestanden niet.

Wat beter kan

- Een **linter** (ESLint/Prettier) om de conventies automatisch af te dwingen, wat de onderhoudbaarheid voor een team verder verhoogt.
- Een **backend** voor een productie-omgeving, zodat sleutels veilig blijven.

Conclusie

Dit project toont de twee kernpunten van het doel aan: ik koos een **programmeeromgeving** (Vite, TypeScript, Bootstrap, localStorage, Spoonacular) die past bij een frontend-dashboard dat meerdere API's combineert, en ik maakte de oplossing **onderhoudbaar voor collega's** door een consistent patroon, getypeerde code, gescheiden lagen en degelijke documentatie.